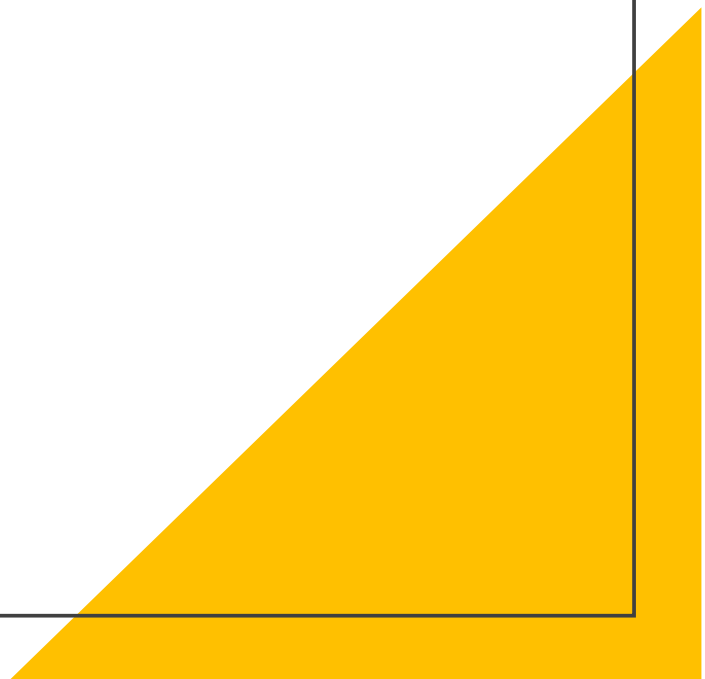


串接Api



什麼是Api？

API(Application Programming Interface)，中文叫應用程式介面，是一種提供不同軟體系統間互動的工具，定義了不同軟體間的互動規範，API允許不同的應用程式、服務或系統之間能夠共享資訊與功能，以約定好的API 接口實現互聯互通。

簡單來說，API 提供了一組定義好的規則與協定，透過資訊的傳遞，讓開發者可以輕鬆整合與使用不同的服務，從而提高應用程式的功能與效能。

什麼是Api？



API 案例



如何使用Api？

要如何在Angular專案中呼叫Api，首先需要先做到幾件事情。

- 將HttpClient透過依賴注入
- 撰寫HttpClient文件
- 呼叫方法

將HttpClient透過依賴注入

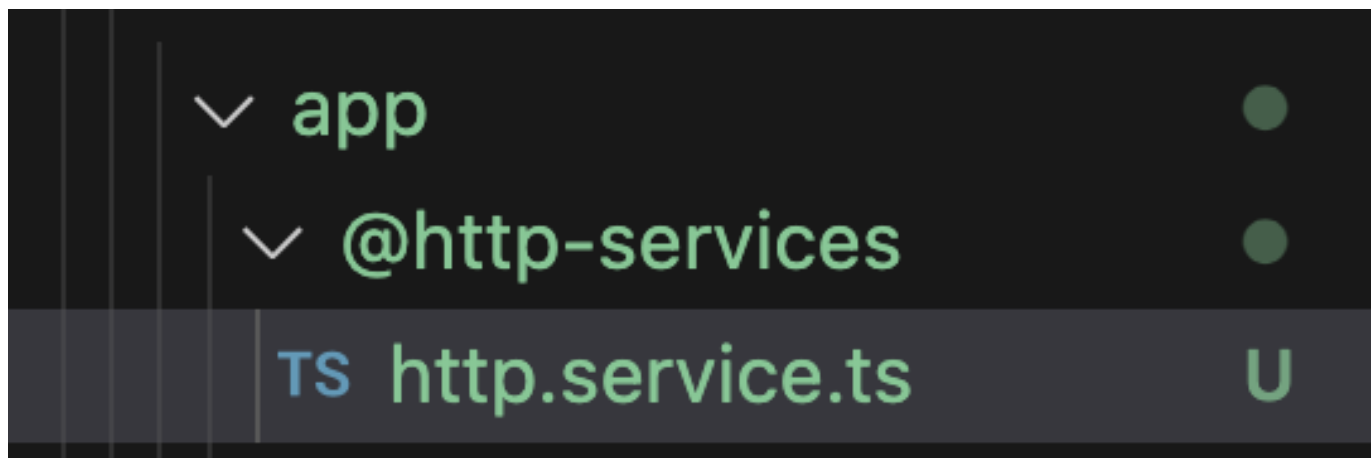
首先先打開你的app.config.ts文件，providers並且在中加入provideHttpClient()，這邊就是在告訴系統我引用了這個並且告訴系統引用的位置在哪裡。

```
import { provideHttpClient } from '@angular/common/http';
```

```
export const appConfig: ApplicationConfig = {  
  providers: [provideZoneChangeDetection({ eventCoalescing: true }),  
             provideRouter(routes),  
             provideHttpClient()]  
};
```

撰寫HttpClient文件

之前有說過建議大家每種文件要放到固定的資料夾中，所以我們這邊先手動建立一個http-service的資料夾並且新增一個檔案http-client.service.ts，這個檔案會是我們後面如果有需要用到串接Api的方法我們都會寫在裡面，這樣可以減少同樣的code一直在不同的地方被重複呼叫。



撰寫HttpClient文件

接下來打開http-client.service.ts並且寫入以下程式，這邊是什麼意思呢？簡單來說撇除export class HttpClientService(命名)之外，上面那邊都只是為了讓系統在網頁一開始打開時就去新建這個檔案放在後台，避免每次要使用時候需要去new這個檔案才能使用。

```
import { Injectable } from "@angular/core";

@Injectable({
  providedIn: 'root'
})

export class HttpClientService {
}
```

撰寫HttpClient文件

接下來將HttpClient注入這個ts中，然後我們就可以開始寫方法了。

```
✓ import { HttpClient } from "@angular/common/http";
```

```
export class HttpClientService {  
  constructor(private http: HttpClient) {}  
}
```


撰寫HttpClient文件

接下來將HttpClient注入這個ts中，然後我們就可以開始寫方法了。

一開始可以先將這四種基礎的方法寫進去，這邊的寫法就是我們會將注入的http裡面的方法回傳給呼叫這個方法的人，用這樣來達到統一處理。

這邊會看到方法後面有(url: string)這個跟後端一樣是指呼叫這個方法時需傳入的數據與數據的類型。

```
// 讀取
getApi(url: string) {
    return this.httpClient.get(url);
}

// 新增
postApi(url: string, postData: any) {
    return this.httpClient.post(url, postData);
}

// 更新
putApi(url: string, putData: any) {
    return this.httpClient.put(url, putData);
}

// 刪除
delApi(url: string) {
    return this.httpClient.delete(url);
}
```

呼叫方法

當你完成上面兩部之後，你就可以開始在你要呼叫的ts去呼叫剛剛寫的HttpClient文件裡面的方法，你需要先在你要呼叫得ts中注入HttpClient文件(他的名稱)並且為他取一個名稱讓下面方法可以指定。

```
import { RouterOutlet } from '@angular/router';  
import { HttpClientService } from '../http-service/http-client.service';
```

```
constructor(private http: HttpClientService) {  
}
```

呼叫方法

接著你就可以在你的方法中去呼叫，範例是呼叫get得方法所以我呼叫getApi方法時只需要傳入url，接下來subscribe(訂閱)的用法就是我們呼叫這個方法時會等getApi回傳才去處理裡面的動作(res)就是指回傳的內容res可自行命名。

```
ngOnInit(): void {  
  this.http.getApi('https://api.freeapi.app/api/v1/public/randomusers')  
    .subscribe((res) => console.log(res));  
}
```

呼叫方法(post/put)

跟get不同post跟put是前端呼叫Api時需要回傳內容給後端，所以我們在呼叫post或者put的Api前會先將要回傳的內容先打包成JSON格式(大部分回傳跟接收Api的資料格式皆為JSON)。

```
ngOnInit(): void {  
  //在呼叫post api之前先把要傳遞的資料打包  
  let postData = {  
    userName: 'Allen',  
    userAge: 18  
  }  
  
  // 呼叫post的方法 第一個值帶api Url 第二個帶要傳遞的值(post一定要帶值 如果真的沒有內容也可以給他一個空{})  
  this.httpClientService  
    .postApi('https://api.freeapi.app/api/v1/kitchen-sink/http-methods/post', postData)  
    .subscribe((res: any) => {  
      console.log(res);  
    })  
}
```

呼叫方法(post/put)

可以試著呼叫<https://api.freeapi.app/api/v1/kitchen-sink/http-methods/post>這隻免費的post Api如果有收到res內容就代表成功了。

```
ngOnInit(): void {  
  //在呼叫post api之前先把要傳遞的資料打包  
  let postData = {  
    userName: 'Allen',  
    userAge: 18  
  }  
  
  // 呼叫post的方法 第一個值帶api Url 第二個帶要傳遞的值(post一定要帶值 如果真的沒有內容也可以給他一個空{})  
  this.httpClientService  
    .postApi('https://api.freeapi.app/api/v1/kitchen-sink/http-methods/post', postData)  
    .subscribe((res: any) => {  
      console.log(res);  
    })  
}
```

免費Api網址

<https://freeapi.hashnode.space/api-guide/apireference/getUsers>

這個網址有免費的Api可以呼叫，請各位試著去嘗試呼叫並且結合畫面顯示將內容顯示出來。



Api Reference	
Send HTML template response	GET
Send XML response	GET
Send gzip response	GET
Send brotli response	GET
Get cookies	GET
Set cookie	POST
Remove cookie	DEL
Redirect to url	GET
Send JPEG image	GET
Send JPG image	GET
Send PNG image	GET
Send WEBP image	GET
Send SVG image	GET

練習

`https://opendata.cwa.gov.tw/api/v1/rest/datastore/F-D0047-065?
Authorization=CWA-69B5A9F7-1D8D-495E-A1F2-
C160E39B4D44&limit=10&format=JSON`

將上面這段Api的內容呈現在畫面上，並且做出畫面(非單純顯示資料)與樣式設計。

结束

